Reimplementing Core Components of Llama 2

Phan Hoang Dung - 23020346
Ngo Quang Dung - 23020344
Truong Trong Duc - 23020360

Institute of Artificial Intelligence

1 Overview

This assignment requires students to independently reimplement the core components of the Llama 2 language model-one of the leading open-source large language models (LLMs) today. Through this process, students will gain a solid understanding of modern Transformer architecture and the training pipeline for text classification models. The assignment uses `stories42M.pt`, a compact Llama 2 model with 42 million parameters, consisting of 8 Transformer layers, pre-trained on the TinyStories dataset.

2 Implementation

2.1 RMSNorm (`llama.py`)

Root Mean Square normalization is computed by scaling x with the inverse square root of the mean of the squared elements. The implementation uses `x.pow(2).mean(-1, keepdim=True)` combined with `torch.rsqrt` for efficient normalization. A small constant `self.eps` is added to avoid division by zero, and the result is scaled by the learnable parameter `self.weight`.

2.2 Rotary Position Embedding (`rope.py`)

RoPE encodes relative positional information by rotating pairs of elements in the query/key vectors with an angle proportional to the token position. The implementation separates real and imaginary parts using `unbind`, computes the rotation angles via `torch.outer`, broadcasts cos and sin to the correct shape, and applies the complex rotation:

$$q_r \cdot \cos - q_i \cdot \sin \quad \text{and} \quad q_r \cdot \sin + q_i \cdot \cos$$

2.3 Scaled Dot-Product Attention (`llama.py`)

Attention scores are computed using QK^T, scaled by $1/\sqrt{d_k}$, followed by softmax and dropout. The result is then multiplied by V to obtain the context vector. All heads are processed in parallel.

2.4 Transformer Block (`llama.py`)

A pre-norm architecture is used with two sub-layers:

$$h = x + \text{Attention}(\text{RMSNorm}(x))$$

followed by:

$$\text{out} = h + \text{FFN}(\text{RMSNorm}(h))$$

Residual connections in both steps help stabilize gradient flow in deep networks.

2.5 Text Continuation (`llama.py`)

With temperature = 0.0, greedy decoding (`argmax`) is used.

With temperature > 0.0, logits are divided by the temperature before applying softmax, and sampling is performed using `torch.multinomial`. Generated tokens are appended to the sequence iteratively until reaching `max_new_tokens`.

2.6 Classification Head (`classifier.py`)

The model calls `self.llama(input_ids)` to obtain hidden states, extracts the last position `[:, -1, :]` as the sentence representation, applies dropout, then feeds it through a Linear layer dim → num_labels, followed by log-softmax.

2.7 AdamW (`optimizer.py`)

First and second moments are updated using β_1 and β_2. Bias correction is applied when `correct_bias = True` to compensate for zero initialization. Parameters are updated using:

$$\theta \leftarrow \theta - \text{step_size} \times \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

Then, decoupled weight decay is applied separately:

$$\theta \leftarrow \theta - \text{lr} \times \lambda \times \theta$$

This is the key difference compared to standard Adam.

2.8 Hyperparameter Grid Search

To further improve model performance during fine-tuning, we conduct a systematic grid search over key hyperparameters, including learning rate, number of training epochs, and dropout rate. Specifically, we explore combinations of learning rate $\in \{1e-5, 2e-5, 5e-5\}$, epochs $\in \{5, 10\}$, and dropout $\in \{0.1, 0.3\}$.

This process allows us to identify the most effective configuration for each dataset and provides insights into how different hyperparameters affect model generalization and stability.

3 Experiment

3.1 Text Continuation

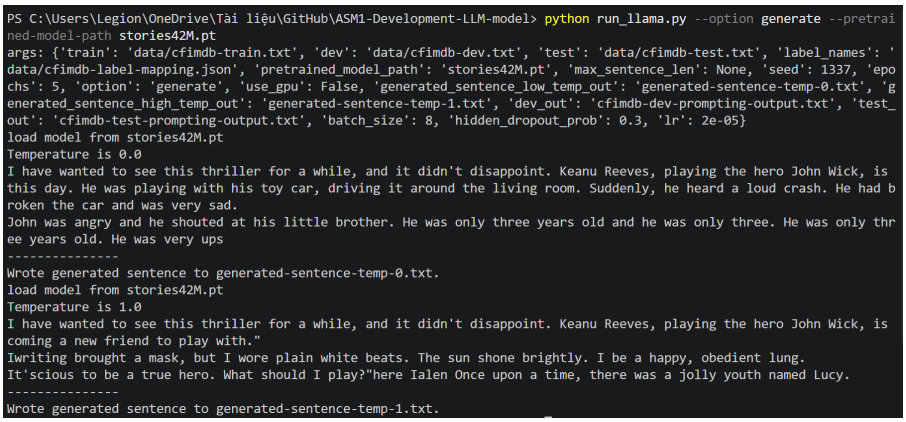
With temperature = 0.0 (greedy decoding), the model always selects the token with the highest probability, resulting in deterministic outputs. The generated sentences are coherent and repeatable across runs.

With temperature = 1.0 (sampling), the model samples from the full probability distribution, producing more diverse outputs.

In both cases, the generated English is grammatically correct, though the content often deviates from the original prompt because the model was pre-trained only on children's stories and lacks context about action movies. Results are saved in generated-sentence-temp-0.txt and generated-sentence-temp-1.txt.

3.2 Zero-shot Prompting

As expected, performance is quite poor on both datasets. Since the model was pre-trained on children's stories, it lacks semantic understanding of movie reviews, making predictions nearly random. This serves as a baseline for comparison with fine-tuning.



```
PS C:\Users\Legion\OneDrive\Tài liệu\GitHub\ASM1-Development-LLM-model> python run_llama.py --option generate --pretrai
ned-model-path stories42M.pt
args: {'train': 'data/cfimdb-train.txt', 'dev': 'data/cfimdb-dev.txt', 'test': 'data/cfimdb-test.txt', 'label_names': '
data/cfimdb-label-mapping.json', 'pretrained_model_path': 'stories42M.pt', 'max_sentence_len': None, 'seed': 1337, 'epo
chs': 5, 'option': 'generate', 'use_gpu': False, 'generated_sentence_low_temp_out': 'generated-sentence-temp-0.txt', 'g
enerated_sentence_high_temp_out': 'generated-sentence-temp-1.txt', 'dev_out': 'cfimdb-dev-prompting-output.txt', 'test_
out': 'cfimdb-test-prompting-output.txt', 'batch_size': 8, 'hidden_dropout_prob': 0.3, 'lr': 2e-05}
load model from stories42M.pt
Temperature is 0.0
I have wanted to see this thriller for a while, and it didn't disappoint. Keanu Reeves, playing the hero John Wick, is
this day. He was playing with his toy car, driving it around the living room. Suddenly, he heard a loud crash. He had b
roken the car and was very sad.
John was angry and he shouted at his little brother. He was only three years old and he was only three. He was only thr
ee years old. He was very ups
---------------
Wrote generated sentence to generated-sentence-temp-0.txt.
load model from stories42M.pt
Temperature is 1.0
I have wanted to see this thriller for a while, and it didn't disappoint. Keanu Reeves, playing the hero John Wick, is
coming a new friend to play with."
Iwriting brought a mask, but I wore plain white beats. The sun shone brightly. I be a happy, obedient lung.
It'scious to be a true hero. What should I play?"here Ialen Once upon a time, there was a jolly youth named Lucy.
---------------
Wrote generated sentence to generated-sentence-temp-1.txt.
```

Figure 1: Output result for text continuation

Dataset	Dev Acc	Test Acc
SST-5	0.215	0.223
CFIMDB	0.502	0.213

Table 1: Zero-shot Prompting Results

3.3 Fine-tuing

We first evaluated the model using default hyperparameters for both datasets:

- **SST-5:** lr = 2e-5, epochs = 5, dropout = 0.3, batch = 64.
- **CFIMDB:** lr = 2e-5, epochs = 5, dropout = 0.3, batch = 8.

Dataset	Dev Acc	Test Acc
SST-5	0.421	0.402
CFIMDB	0.857	0.469

Table 2: Fine-tuning Results (Default Parameters)

And then We performed a grid search over 12 hyperparameter combinations:

- **Learning Rate:** $\{1\text{e-}5, 2\text{e-}5, 5\text{e-}5\}$
- **Epochs:** $\{5, 10\}$
- **Dropout:** $\{0.1, 0.3\}$

SST-5 Results (batch size 64): The best configuration achieved a dev accuracy of 0.4223 and test accuracy of 0.4149 with $\text{lr} = 1\text{e-}5, \text{epochs} = 10, \text{dropout} = 0.3$. Overall, performance across configurations was relatively consistent, ranging from 0.396 to 0.422 on the dev set. A lower learning rate combined with more epochs yields the most stable results. While the improvement over zero-shot is significant, the 5-class classification task remains challenging for this model size.

CFIMDB Results (batch size 8): The best configuration achieved a high dev accuracy of 0.9143 ($\text{lr} = 1\text{e-}5, \text{epochs} = 10, \text{dropout} = 0.1$). However, test accuracy only reached 0.5164, indicating severe overfitting. The model performs exceptionally well on the small dev set but fails to generalize to the test set. This is likely due to the small dataset size and low batch size, making the model more prone to overfitting compared to SST-5.

Dataset	lr	Epochs	Dropout	Dev Acc	Test Acc
SST-5	1e-5	10	0.3	0.4223	0.4149
CFIMDB	1e-5	10	0.1	0.9143	0.5164

Table 3: Fine-tuning Results (Best Hyperparameters)

4 Conclusion

This assignment helped the team gain a deep understanding of the architecture and mechanisms of the Llama 2 language model by implementing its core components from scratch. These components include RMSNorm, Rotary Position Embedding, Scaled Dot-Product Attention, Transformer Blocks, text generation mechanisms, and the AdamW optimizer - all of which passed the required unit tests.

The three experiments, conducted in increasing order of complexity, clearly demonstrate the differences between approaches. Text generation confirms that the model functions correctly after loading pre-trained weights. Zero-shot prompting produces near-random results, showing that even a powerful model cannot perform well on specialized tasks without adaptation. Fine-tuning addresses this limitation and significantly improves performance, especially on CFIMDB with a dev accuracy of 0.9143.

However, the results also reveal a key limitation: the large gap between dev and test accuracy on CFIMDB indicates severe overfitting when training on small datasets with low batch sizes. On SST-5, accuracy plateaus around 0.42, reflecting the inherent difficulty of 5-class classification with a relatively small model.

From this assignment, two important practical lessons emerge:

- A small learning rate (1e-5) combined with more training epochs provides the most stable fine-tuning strategy for large language models.
- Implementing and testing individual components independently is an effective approach for building and debugging complex deep learning systems.